

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	N.AKHIL KUMAR	Reg.No.:	192425128
Department:	AI&ML	Date:	14/08/2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Answer to Question 1: Signed 8-bit 2's Complement Arithmetic

Given: +45 and -23. In 8-bit 2's complement form, +45 = 00101101. To obtain -23, first write +23 = 00010111, invert the bits and add 1.

```
00010111 (+23)
11101000 (1's complement)
+ 00000001
-----
11101001 (-23)

Addition: (+45) + (-23)
  00101101 (+45)
+ 11101001 (-23)
-----
1 00010110 (discard the carry out)
```

Therefore, $00010110 = 16 + 4 + 2 = +22$. There is no overflow because the operands have different signs; signed 2's complement addition can overflow only when two operands of the same sign produce a result of the opposite sign.

Subtraction: (+45) - (-23)

Use $A - B = A + (2\text{'s complement of } B)$. Hence $45 - (-23) = 45 + 23$, and $+23 = 00010111$.

```

00101101 (+45)
+ 00010111 (+23)
-----
01000100

```

Therefore, $01000100 = 64 + 4 = +68$. The signed 8-bit range is -128 to +127, so no overflow occurs.

Operation	Binary result	Decimal result	Overflow
$+45 + (-23)$	00010110	+22	No
$+45 - (-23)$	01000100	+68	No

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Answer to Question 2: 4-bit Carry Look-Ahead Adder (CLA)

Use $A = 1011$, $B = 0110$ and $C_0 = 0$. For each bit, $P_i = A_i \text{ XOR } B_i$ and $G_i = A_i \text{ AND } B_i$. Carries are generated in advance by $C_{i+1} = G_i + P_i C_i$.

Bit i	A_i	B_i	P_i	G_i
0	1	0	1	0
1	1	1	0	1
2	0	1	1	0
3	1	0	1	0

$$\begin{aligned}
C1 &= G0 + P0C0 = 0 + (1)(0) = 0 \\
C2 &= G1 + P1G0 + P1P0C0 = 1 \\
C3 &= G2 + P2G1 + P2P1G0 + P2P1P0C0 = 1 \\
C4 &= G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0C0 = 1
\end{aligned}$$

$$S0 = P0 \text{ XOR } C0 = 1; S1 = 0; S2 = 0; S3 = 0$$

$$\begin{array}{r}
1011 \\
+ 0110 \\
\hline
1\ 0001
\end{array}$$

The four-bit sum is 0001 and the carry-out is 1, giving 10001 (17 decimal). In a ripple carry adder, every stage waits for the preceding carry, whereas a CLA produces C1 to C4 from the generate/propagate expressions in parallel. Thus, CLA has lower carry delay, although it uses more logic hardware.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Answer to Question 3: Booth's Algorithm for (-6) x (+5)

Use 4-bit 2's complement: M = 1010 (-6), Q = 0101 (+5), A = 0000 and Q-1 = 0. Booth rules: 01 means A = A + M; 10 means A = A - M; 00 and 11 mean no arithmetic operation. An arithmetic right shift follows every iteration.

Iteration n	Q0Q-1	Operation before shift	A, Q, Q-1 after shift
Initial	-	A = 0000; Q = 0101; Q-1 = 0	0000, 0101, 0
1	10	A = 0000 - 1010 = 0110	0011, 0010, 1
2	01	A = 0011 + 1010 = 1101	1110, 1001, 0
3	10	A = 1110 - 1010 = 0100	0010, 0100, 1
4	01	A = 0010 + 1010 = 1100	1110, 0010, 0

Final product AQ = 11100010

```

11100010
00011101 (1's complement)
+ 00000001
-----
00011110 = 30

```

The leading bit is 1, so the product is negative: 11100010 = -30. Therefore, (-6) x (+5) = -30. Booth's algorithm handles signed values naturally and reduces additions/subtractions when the multiplier contains runs of 1s.

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Answer to Question 4: Restoring and Non-Restoring Division of 13 by 3

Use Q = 1101 (13), M = 0011 (3) and A = 0000. Four iterations are required.

A. Restoring division

Step	After left shift (A,Q)	A - M	Decision / resulting Q
1	0001, 1010	1110 (negative)	Restore A = 0001; Q = 1010
2	0011, 0100	0000 (non-negative)	Set Q0 = 1: A = 0000; Q = 0101
3	0000, 1010	1101 (negative)	Restore A = 0000; Q = 1010
4	0001, 0100	1110 (negative)	Restore A = 0001; Q = 0100

Restoring result: quotient Q = 0100 = 4 and remainder A = 0001 = 1. Check: $3 \times 4 + 1 = 13$.

B. Non-restoring division

For non-restoring division, if the previous partial remainder is non-negative, shift and subtract M. If it is negative, shift and add M. A negative partial remainder is not restored immediately.

Step	Previous A	Shifted A,Q	Operation / new A	Q after Q0
1	Non-negative	0001, 1010	$0001 - 0011 = 1110$	1010 (0)

2	Negative	1101, 0100	$1101 + 0011 = 0000$	0101 (1)
3	Non-negative	0000, 1010	$0000 - 0011 = 1101$	1010 (0)
4	Negative	1011, 0100	$1011 + 0011 = 1110$	0100 (0)

The final A is negative, so apply correction: $1110 + 0011 = 0001$. Thus the non-restoring result is also quotient $0100 = 4$, remainder $0001 = 1$. Non-restoring division generally requires fewer intermediate restoration additions and is therefore more efficient in high-speed hardware.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Answer to Question 5: IEEE 754 Representation of -13.25

$$13.25 = 1101.01_2$$

$$-13.25 = -1.10101 \times 2^3$$

The number is negative, so the sign bit is 1. The actual exponent is 3. The fraction field stores 10101 followed by zeros; the leading normalized 1 is implicit.

Single precision (32 bits)

Field	Value	Reason
Sign (1 bit)	1	Negative number
Exponent (8 bits)	10000010	$3 + \text{bias } 127 = 130$
Fraction (23 bits)	10101000000000000000000	Bits after the implicit leading 1

$$1 \mid 10000010 \mid 10101000000000000000000$$

$$= 1100000101010100000000000000000$$

Double precision (64 bits)

Field	Value	Reason
Sign (1 bit)	1	Negative number
Exponent (11 bits)	10000000010	$3 + \text{bias } 1023 = 1026$

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)